Análisis del Proyecto LFPlayer

Fecha: 5 de noviembre de 2026
Proyecto: lfplayer - Reproductor de música personalizado

🔴 Problemas Críticos

1. Layout obtiene TODAS las canciones sin paginación

Archivo: `app/app/layout.tsx:46-64`

```
// Obtiene toda la biblioteca en cada carga - fallará con 10k+ canciones
while (!finished) {
  const { data: batchSongs } = await supabase.from("songs").select("*").range(from, to);
}
```

Impacto: Carga completa de la base de datos en cada render del servidor. Memoria y tiempo de respuesta crecen linealmente con el número de canciones.

Solución: Mover a Server Component con paginación o usar `refetchSongs` solo del lado cliente con infinite scroll.

---

2. Componente masivo (God Component)

Archivo: `components/music-library.tsx` - **1002 líneas, 30+ useState**

Problemas:

- Viola principio de responsabilidad única
- Toda la lógica de admin, descargas, limpieza, diálogos en un componente
- Imposible de testear, mantener, depurar

Dividir en:

- `MusicLibrary` - vista principal
- `AdminPanel` - panel de administrador
- `DownloadManager` - gestión de descargas
- `CleanupTools` - herramientas de limpieza
- `ArtistAccordion` - acordeón por artista
- `SongGrid` - grid de canciones virtualizado

---

3. Player de audio con elementos duplicados

Archivo: `components/music-player.tsx:151-169, 219-237`

- Dos elementos `<audio>` separados para layouts "card" y "bar"
- Múltiples `useEffect` para sincronización causan race conditions
- Lógica de play/pause/seek duplicada

Solución: Un solo elemento `<audio>` con ref, diferencia de layout solo vía CSS.

🟠 Alta Prioridad

4. Re-renders innecesarios en Context

Archivo: `contexts/MusicLibraryContext.tsx:95-101`

```
const contextValue = { songs, genres, playlists, refetchSongs, fetchPlaylists };
// Nuevo objeto en cada render → todos los consumers re-renderizan
```

Solución: `useMemo` en context value, separar contextos por frecuencia de actualización (songs vs playlists vs genres).

---

5. Sin cache cliente (SWR/React Query)

- Cada navegación hace refetch via `router.refresh()`
- No hay optimistic updates para borrados/creación playlists

- Experiencia de usuario lenta, uso de red innecesario

Solución: Integrar `@tanstack/react-query` o SWR.

---

6. Verificación admin por email string (Hardcoded)

Archivos: `app/app/layout.tsx:24`, `app/api/genres/route.ts:63`

```
if (user.email?.toLowerCase().includes('lucidio')) // Frágil, inseguro
```

Problemas:

- No escalable
- Inseguro (cualquier email con "lucidio" pasa)
- No sigue patrón RBAC estándar

Solución: Usar columna `role` en tabla `profiles` con valores `admin`/`user`/`guest`.

---

7. Estado volume como array incorrecto

Archivo: `components/music-player.tsx:33`

```
const [volume, setVolume] = useState([100]) // ¿Por qué array?
```

Solución: `useState<number>(100)` con tipado correcto.

🟡 Media Prioridad

8. Sin Error Boundaries

- No hay degradación graceful para fallos de audio
- Error en un componente rompe toda la app

9. Gaps de Type Safety

```
playlists: any[] // MusicLibraryContext.tsx:10
```

Solución: Definir interfaz `Playlist` en `lib/types.ts`.

10. Sin virtualización en grids

- Lag con 500+ canciones renderizadas
- **Solución:** `@tanstack/react-virtual` o `react-window`

11. Idiomas mezclados

- Español/Inglés en comentarios, variables, strings UI
- **Solución:** Estandarizar a español (proyecto en español) o inglés con i18n

12. Duplicación API limpieza

- 3 endpoints similares: `/api/cleanup-supabase`, `/api/cleanup-supabase/broken-links`, `/api/cleanup`
- **Solución:** Unificar en un solo endpoint con acciones parametrizadas

---

🟢 Quick Wins (Fáciles y alto impacto)

Problema	Archivo	Solución
`formatTime` duplicado	`song-card.tsx:19`, `music-player.tsx:94`	Mover a `lib/utils.ts`
Archivos debug en root	`check*.js`, `repair*.js`, `*.json`	Mover a `scripts/` o borrar
Sin config ESLint	`package.json:8`	Añadir `.eslintrc.json` con reglas Next.js
Orden géneros hardcodeado	`genres/route.ts:15-19`	Usar columna `display_order` de BD
Sin tests	—	Añadir Vitest + React Testing Library
Secrets en `.env.local`	`.env.local`	Verificar `.gitignore` excluye correctamente

Prioridad de Refactor Sugerida

Fase	Tarea	Esfuerzo	Impacto
1	Dividir `music-library.tsx` en 5+ componentes	Alto	Muy Alto
2	Añadir paginación/infinite scroll a canciones	Medio	Alto
3	Arreglar audio player (un solo elemento)	Medio	Alto
4	Integrar React Query para data fetching	Medio	Alto
5	Memoizar context values + separar contextos	Bajo	Alto
6	RBAC real en lugar de match por email	Bajo	Alto (Seguridad)
7	Añadir Error Boundaries + skeletons	Bajo	Medio
8	TypeScript strict mode - arreglar `any`	Medio	Medio

Arquitectura Actual vs Propuesta

Actual (Problemática)

```
app/app/layout.tsx (Server Component)
  └─ Fetch ALL songs + genres (sin paginación)
       └─ MusicLibraryProvider (Client)
            └─ music-library.tsx (1000+ líneas, TODO)
                 └─ MusicPlayerContext (Client)
                      └─ music-player.tsx (duplicado audio)
```

Propuesta (Escalable)

```
app/app/layout.tsx (Server Component)
  └─ Fetch genres only (liviano)
       └─ MusicLibraryProvider (Client) → songs via React Query
            ├─ MusicLibrary (vista principal)
            │    ├─ GenreFilter
            │    ├─ ArtistAccordion (virtualizado)
            │    │    └─ SongGrid (virtualizado)
            │    └─ GlobalMusicPlayer
            ├─ AdminPanel (solo si admin)
            │    ├─ DownloadManager
            │    └─ CleanupTools
            └─ MusicPlayerContext (optimizado)
                 └─ music-player.tsx (un solo audio)
```

Métricas de Código Actual

Métrica	Valor	Objetivo
Líneas `music-library.tsx`	1,002	< 200
`useState` en un componente	30+	< 10
Context providers anidados	3	2-3 (separados)
Archivos debug en root	15+	0
Cobertura de tests	0%	> 70%
TypeScript `any` usages	~15	0

Próximos Pasos Inmediatos

1. **Hoy:** Mover archivos debug a `scripts/` o borrar
2. **Esta semana:** Dividir `music-library.tsx` en componentes
3. **Esta semana:** Añadir paginación a fetch de canciones
4. **Próxima semana:** Integrar React Query
5. **Próxima semana:** Arreglar audio player y RBAC

Generado automáticamente tras análisis de código estático